

钢铁夹杂物析出相计算与可视化系统，部分源代码：

```
# ===== File: run.py =====
# -*- coding: utf-8 -*-
"""开发模式入口: python run.py"""
import sys
from pathlib import Path

# 确保 backend 在 sys.path 中
backend_dir = Path(__file__).resolve().parent / "backend"
if str(backend_dir) not in sys.path:
    sys.path.insert(0, str(backend_dir))

import uvicorn
from app.config import settings

if __name__ == "__main__":
    uvicorn.run(
        "app.main:app",
        host=settings.server_host,
        port=settings.server_port,
        reload=True,
        reload_dirs=[str(backend_dir / "app")],
    )

# ===== File: build.py =====
# -*- coding: utf-8 -*-
"""PyInstaller 打包脚本: 生成可迁移的 onedir 运行目录."""
from __future__ import annotations

import shutil
import subprocess
import sys
import urllib.request
from pathlib import Path

ROOT = Path(__file__).resolve().parent
DIST_NAME = "FactSage_InclPrecip"
BACKEND = ROOT / "backend"
FRONTEND = ROOT / "frontend"
DIST_ROOT = ROOT / "dist"
DIST_DIR = DIST_ROOT / DIST_NAME
BUILD_ROOT = DIST_ROOT / "_build"
PYINSTALLER_WORK = DIST_ROOT / "_pyinstaller_work"
PYINSTALLER_SPEC = DIST_ROOT / "_pyinstaller_spec"
ENTRY = BUILD_ROOT / "start.py"
STAGED_FRONTEND = BUILD_ROOT / "frontend"

ECHARTS_URL = "https://cdn.jsdelivr.net/npm/echarts@5/dist/echarts.min.js"
ECHARTS_REL = Path("vendor") / "echarts.min.js"
ECHARTS_CDN_SNIPPET = (
    '<script src="https://cdn.jsdelivr.net/npm/echarts@5/dist/echarts.min.js"></script>'
)
```

```

)
ECHARTS_LOCAL_SNIPPET = '<script src="vendor/echarts.min.js"></script>'

def _remove_tree(path: Path) -> None:
    if path.exists():
        shutil.rmtree(path)

def _write_entry() -> None:
    """生成 PyInstaller 入口, 放在 dist 临时目录, 避免污染源码树。"""
    ENTRY.parent.mkdir(parents=True, exist_ok=True)
    ENTRY.write_text(
        '# -*- coding: utf-8 -*-\n'
        '"""Frozen service entrypoint."""\n'
        'import uvicorn\n'
        'from app.config import settings\n'
        'from app.main import app\n\n'
        'if __name__ == "__main__":\n'
        '    uvicorn.run(app, host=settings.server_host, port=settings.server_port)\n',
        encoding="utf-8",
    )

def _stage_frontend() -> None:
    """复制前端并替换 CDN 依赖, 确保打包产物离线可用。"""
    _remove_tree(STAGED_FRONTEND)
    shutil.copytree(FRONTEND, STAGED_FRONTEND)

    vendor_dir = STAGED_FRONTEND / ECHARTS_REL.parent
    vendor_dir.mkdir(parents=True, exist_ok=True)

    source_echarts = FRONTEND / ECHARTS_REL
    staged_echarts = STAGED_FRONTEND / ECHARTS_REL
    if source_echarts.exists():
        shutil.copy2(source_echarts, staged_echarts)
    else:
        print(f"下载前端离线依赖: {ECHARTS_URL}")
        urllib.request.urlretrieve(ECHARTS_URL, staged_echarts)

    index_path = STAGED_FRONTEND / "index.html"
    index_html = index_path.read_text(encoding="utf-8-sig")
    if ECHARTS_CDN_SNIPPET in index_html:
        index_html = index_html.replace(ECHARTS_CDN_SNIPPET, ECHARTS_LOCAL_SNIPPET)
    elif ECHARTS_LOCAL_SNIPPET not in index_html:
        raise RuntimeError("index.html 中未找到 ECharts 脚本引用, 无法保证离线前端。")
    index_path.write_text(index_html, encoding="utf-8", newline="\n")

def _copy_runtime_assets() -> None:

```

```

"""复制运行时可变/可配置资产到 exe 同级目录。"""
shutil.copy2(BACKEND / "config.json", DIST_DIR / "config.json")

for source, target_name in (
    (BACKEND / "templates", "templates"),
    (STAGED_FRONTEND, "frontend"),
):
    target = DIST_DIR / target_name
    _remove_tree(target)
    shutil.copytree(source, target)

demo_dir = ROOT / "demo"
packaged_demo = DIST_DIR / "demo"
_remove_tree(packaged_demo)
packaged_demo.mkdir(parents=True, exist_ok=True)
for name in ("Equi2.res", "Equ2222i.res"):
    source = demo_dir / name
    if source.exists():
        shutil.copy2(source, packaged_demo / name)

for dirname in ("data", "work", "results"):
    (DIST_DIR / dirname).mkdir(parents=True, exist_ok=True)

def _run_pyinstaller() -> None:
    cmd = [
        sys.executable,
        "-m",
        "PyInstaller",
        "--noconfirm",
        "--clean",
        "--onedir",
        "--name",
        DIST_NAME,
        "--distpath",
        str(DIST_ROOT),
        "--workpath",
        str(PYINSTALLER_WORK),
        "--specpath",
        str(PYINSTALLER_SPEC),
        "--paths",
        str(BACKEND),
        "--hidden-import",
        "uvicorn.logging",
        "--hidden-import",
        "uvicorn.protocols.http.auto",
        "--hidden-import",
        "uvicorn.protocols.websockets.auto",
        "--hidden-import",
        "uvicorn.lifespan.on",
    ]

```

```

        str(ENTRY),
    ]

    print(f"执行: {' '.join(cmd)}")
    subprocess.run(cmd, check=True)

def main() -> None:
    _remove_tree(DIST_DIR)
    _remove_tree(PYINSTALLER_WORK)
    _remove_tree(PYINSTALLER_SPEC)
    _remove_tree(BUILD_ROOT)

    _write_entry()
    _stage_frontend()
    _run_pyinstaller()
    _copy_runtime_assets()

    print(f"\n打包完成: {DIST_DIR}")
    print(f"运行: {DIST_DIR / (DIST_NAME + '.exe')}")
    print("迁移时复制整个目录, 不要只复制 exe。")

if __name__ == "__main__":
    main()

<!-- ===== File: backend/config.json ===== -->
{
    "server": {
        "host": "127.0.0.1",
        "port": 10688
    },
    "factsage": {
        "dir": "C:\\\\FactSage",
        "timeout_seconds": 600
    },
    "mock": {
        "enabled": "auto"
    }
}

# ===== File: backend/app/config.py =====
# -*- coding: utf-8 -*-
"""应用配置 — 从 config.json 加载, 支持环境变量覆盖"""
from __future__ import annotations

import json
import os
import sys
from pathlib import Path

```

```

from typing import Any, Dict

# — 基准目录 —————
# 打包模式 (PyInstaller): exe 所在目录
# 开发模式: backend/ 目录
_IS_FROZEN = getattr(sys, "frozen", False)
_BASE_DIR = (
    Path(sys.executable).resolve().parent
    if _IS_FROZEN
    else Path(__file__).resolve().parent.parent
)

_DEFAULT_CONFIG: Dict[str, Any] = {
    "server": {"host": "127.0.0.1", "port": 10688},
    "factsage": {
        "dir": r"C:\FactSage",
        "exe_name": "EquiSage.exe",
        "timeout_seconds": 600,
    },
    "paths": {
        "work_root": "./work",
        "templates_dir": "templates" if _IS_FROZEN else "./templates",
        "frontend_dir": "frontend" if _IS_FROZEN else "../frontend",
        "db_path": "./data/inclprecip.db",
        "results_dir": "./results",
    },
    "mock": {"enabled": "auto", "delay_seconds": 2.0},
    "species_filter": {
        "default_min_gram": 1e-8,
        "options": [1e-10, 1e-8, 1e-6, 1e-4],
    },
}

def _deep_merge(base: dict, override: dict) -> dict:
    """递归合并, override 中的值覆盖 base"""
    merged = base.copy()
    for k, v in override.items():
        if k in merged and isinstance(merged[k], dict) and isinstance(v, dict):
            merged[k] = _deep_merge(merged[k], v)
        else:
            merged[k] = v
    return merged

class Settings:
    """全局配置: config.json → 环境变量 → 自动检测"""

    def __init__(self, config_path: Path | None = None) -> None:
        cfg_path = config_path or _BASE_DIR / "config.json"

```

```

    if cfg_path.exists():
        with open(cfg_path, "r", encoding="utf-8-sig") as f:
            user_cfg = json.load(f)
    else:
        user_cfg = {}
    self._cfg = _deep_merge(_DEFAULT_CONFIG, user_cfg)

# — 路径解析 —————

def _resolve(self, raw: str) -> Path:
    """相对路径基于 _BASE_DIR 解析, 绝对路径原样返回"""
    p = Path(os.path.expandvars(raw))
    return p if p.is_absolute() else (_BASE_DIR / p).resolve()

# — FactSage —————

@property
def factsage_dir(self) -> Path:
    return Path(os.getenv("FACTSAGE_DIR") or self._cfg["factsage"]["dir"])

@property
def factsage_exe(self) -> Path:
    return self.factsage_dir / self._cfg["factsage"]["exe_name"]

@property
def factsage_timeout(self) -> int:
    return int(self._cfg["factsage"]["timeout_seconds"])

# — 路径 —————

@property
def work_root(self) -> Path:
    return self._resolve(
        os.getenv("WORK_ROOT") or self._cfg["paths"]["work_root"]
    )

@property
def templates_dir(self) -> Path:
    return self._resolve(
        os.getenv("TEMPLATES_DIR") or self._cfg["paths"]["templates_dir"]
    )

@property
def frontend_dir(self) -> Path:
    return self._resolve(
        os.getenv("FRONTEND_DIR") or self._cfg["paths"]["frontend_dir"]
    )

@property
def db_path(self) -> Path:

```

```

        return self._resolve(
            os.getenv("DB_PATH") or self._cfg["paths"]["db_path"]
        )

    @property
    def results_dir(self) -> Path:
        return self._resolve(
            os.getenv("RESULTS_DIR") or self._cfg["paths"]["results_dir"]
        )

# — 服务器 —————

    @property
    def server_host(self) -> str:
        return os.getenv("HOST") or self._cfg["server"]["host"]

    @property
    def server_port(self) -> int:
        return int(os.getenv("PORT") or self._cfg["server"]["port"])

# — Mock —————

    @property
    def mock_mode(self) -> bool:
        val = os.getenv("MOCK_MODE") or self._cfg["mock"]["enabled"]
        if isinstance(val, bool):
            return val
        val = str(val).lower()
        if val == "auto":
            return not self.factsage_exe.exists()
        return val in ("1", "true", "yes")

    @property
    def mock_delay(self) -> float:
        return float(self._cfg["mock"]["delay_seconds"])

    @property
    def default_species_min_gram(self) -> float:
        return float(self._cfg["species_filter"]["default_min_gram"])

    @property
    def species_min_gram_options(self) -> list[float]:
        return [float(v) for v in self._cfg["species_filter"]["options"]]

settings = Settings()

# ===== File: backend/app/models.py =====
# -*- coding: utf-8 -*-
"""Pydantic 数据模型 — API 请求/响应定义"""

```

```

from __future__ import annotations

from enum import Enum
from typing import Optional

from pydantic import BaseModel, Field, field_validator

# — 枚举 —————

class JobStatus(str, Enum):
    pending = "pending"
    running = "running"
    completed = "completed"
    failed = "failed"

# — 请求模型 —————

class ElementMass(BaseModel):
    symbol: str
    mass_g: float = Field(..., ge=0)

class TempRange(BaseModel):
    start_c: float = Field(800, ge=0)
    end_c: float = Field(1600, ge=0)
    step_c: float = Field(10, gt=0)
    pressure_atm: float = Field(1.0, gt=0)

    @field_validator("end_c")
    @classmethod
    def end_gt_start(cls, v, info):
        if "start_c" in info.data and v <= info.data["start_c"]:
            raise ValueError("end_c must be greater than start_c")
        return v

class JobRequest(BaseModel):
    template_id: str
    elements: list[ElementMass]
    temp_range: TempRange = Field(default_factory=TempRange)

    @field_validator("elements")
    @classmethod
    def validate_elements(cls, v):
        symbols = [e.symbol for e in v]
        if len(symbols) != len(set(symbols)):

```



```

        raise ValueError("Duplicate element symbols")
    # 只统计有效质量元素 (排除 1E-10 级别的痕量占位)
    total = sum(e.mass_g for e in v if e.mass_g > 0.001)
    if abs(total - 100.0) > 5.0:
        raise ValueError(f"Total mass must be ~100g, got {total:.2f}g")
    return v

class ChartDataRequest(BaseModel):
    species_names: list[str]
    value_type: str = Field(
        "gram", pattern="^(gram|mole|wt_pct|activity|mole_fraction)$"
    )

# — 响应模型 —————

class SpeciesListItem(BaseModel):
    name: str
    display_name: str
    category: str
    phase: str
    max_gram: float

class ChartSeries(BaseModel):
    name: str
    display_name: str
    data: list[float]

class ChartDataResponse(BaseModel):
    temperatures: list[float]
    series: list[ChartSeries]

class JobResponse(BaseModel):
    job_id: str
    status: JobStatus
    template_id: Optional[str] = None
    created_at: Optional[str] = None
    error: Optional[str] = None

class JobListItem(BaseModel):
    job_id: str
    status: JobStatus
    template_id: str
    created_at: str

```

```

# ===== File: backend/app/main.py =====
# -*- coding: utf-8 -*-
"""FastAPI 应用入口"""
from __future__ import annotations

import logging
from contextlib import asynccontextmanager

from fastapi import FastAPI, Request
from fastapi.exceptions import RequestValidationError
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import FileResponse, JSONResponse, Response
from fastapi.staticfiles import StaticFiles

from .config import settings
from .routers import jobs
from .services.job_manager import job_manager

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
)
logger = logging.getLogger(__name__)

@asynccontextmanager
async def lifespan(app: FastAPI):
    logger.info(
        "启动 FactSage 夹杂物/析出相计算服务 mock=%s port=%d",
        settings.mock_mode, settings.server_port,
    )
    await job_manager.start()
    yield
    await job_manager.stop()

app = FastAPI(
    title="FactSage 夹杂物/析出相计算",
    description="基于 FactSage Equilib 模块的温度扫描计算 API",
    version="1.0.0",
    lifespan=lifespan,
)

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_methods=["*"],
    allow_headers=["*"],
)

```

```

@app.exception_handler(RequestValidationError)
async def validation_exception_handler(request: Request, exc: RequestValidationError):
    logger.error(
        "422 验证失败 url=%s errors=%s", request.url, exc.errors()
    )
    return JSONResponse(status_code=422, content={"detail": exc.errors()})

app.include_router(jobs.router)

@app.get("/favicon.ico", include_in_schema=False)
async def favicon():
    return Response(status_code=204)

_FE = settings.frontend_dir
if _FE.exists():
    for static_name in ("css", "js", "vendor"):
        static_dir = _FE / static_name
        if static_dir.exists():
            app.mount(
                f"/{static_name}",
                StaticFiles(directory=static_dir),
                name=static_name,
            )

    @app.get("/")
    async def index():
        return FileResponse(_FE / "index.html")
else:
    logger.warning("前端目录不存在: %s, 仅提供 API 服务", _FE)

# ===== File: backend/app/routers/jobs.py =====
# -*- coding: utf-8 -*-
"""API 路由: 任务提交 / 查询 / 模板 / 结果数据"""
from __future__ import annotations

import csv
import io
import json
import logging
import zipfile
from typing import List

from fastapi import APIRouter, HTTPException, Query
from fastapi.responses import StreamingResponse

from ..config import settings

```

```

from ..models import (
    ChartDataResponse,
    ChartSeries,
    JobListItem,
    JobRequest,
    JobResponse,
    JobStatus,
    SpeciesListItem,
)
from ..services.job_manager import job_manager
from ..services.res_parser import filter_species_by_min_gram
from ..services.template_renderer import load_registry, load_template_meta

logger = logging.getLogger(__name__)
router = APIRouter(prefix="/api", tags=["jobs"])

# — 模板接口 —————

@router.get("/templates")
async def list_templates():
    """列出所有可用模板"""
    reg = load_registry()
    return [
        {"id": t["id"], "name": t["name"], "description": t.get("description", "")}
        for t in reg.get("templates", [])
    ]

@router.get("/templates/{template_id}")
async def get_template(template_id: str):
    """获取模板详情（含元素列表和默认参数）"""
    try:
        meta = load_template_meta(template_id)
    except FileNotFoundError:
        raise HTTPException(status_code=404, detail=f"模板 '{template_id}' 不存在")
    return meta

# — 计算接口 —————

@router.post("/calculate")
async def calculate(request: JobRequest) -> JobResponse:
    """提交一次计算任务"""
    # 检查模板存在
    try:
        load_template_meta(request.template_id)
    except FileNotFoundError:

```

```

        raise HTTPException(
            status_code=400, detail=f"模板 '{request.template_id}' 不存在"
        )

    job_id = await job_manager.submit(request)
    job = job_manager.get(job_id)
    return JobResponse(
        job_id=job_id,
        status=job["status"],
        template_id=job["template_id"],
        created_at=job["created_at"],
    )

@router.get("/jobs/{job_id}")
async def get_job(job_id: str) -> JobResponse:
    """查询任务状态"""
    job = job_manager.get(job_id)
    if not job:
        raise HTTPException(status_code=404, detail="任务不存在")
    return JobResponse(
        job_id=job["job_id"],
        status=job["status"],
        template_id=job["template_id"],
        created_at=job["created_at"],
        error=job["error"],
    )

@router.get("/jobs/{job_id}/species")
async def get_species(
    job_id: str,
    min_gram: float = Query(
        settings.default_species_min_gram,
        ge=0,
        description="按物种最大质量过滤, 单位 g",
    ),
) -> List[SpeciesListItem]:
    """获取物种列表（轻量，不含数据序列）"""
    result = job_manager.get_parsed_result(job_id)
    if result is None:
        job = job_manager.get(job_id)
        if not job:
            raise HTTPException(status_code=404, detail="任务不存在")
        if job["status"] != JobStatus.completed:
            raise HTTPException(status_code=400, detail=f"任务状态: {job['status'].value}")
        raise HTTPException(status_code=404, detail="结果文件不存在")

    items = []
    for s in filter_species_by_min_gram(result["species"], min_gram):

```

```

        items.append(SpeciesListItem(
            name=s["name"],
            display_name=s["display_name"],
            category=s["category"],
            phase=s["phase"],
            max_gram=s["max_gram"],
        ))
    items.sort(key=lambda x: x.max_gram, reverse=True)
    return items

@router.get("/jobs/{job_id}/chart-data")
async def get_chart_data(
    job_id: str,
    species_names: str = Query(..., description="逗号分隔的物种名称"),
    value_type: str = Query("gram", description="数据类型: gram/mole/wt_pct/activity/mole_fraction"),
) -> ChartDataResponse:
    """获取选中物种的图表数据"""
    result = job_manager.get_parsed_result(job_id)
    if result is None:
        raise HTTPException(status_code=404, detail="结果不存在")

    names = [n.strip() for n in species_names.split(",") if n.strip()]
    if not names:
        raise HTTPException(status_code=400, detail="至少选择一个物种")

    # 值类型映射
    type_map = {
        "gram": "grams",
        "mole": "moles",
        "wt_pct": "wt_pcts",
        "activity": "activities",
        "mole_fraction": "mole_fractions",
    }
    data_key = type_map.get(value_type)
    if not data_key:
        raise HTTPException(status_code=400, detail=f"无效的值类型: {value_type}")

    # 构建 species 索引
    species_map = {s["name"]: s for s in result["species"]}

    series = []
    for name in names:
        s = species_map.get(name)
        if s is None:
            continue
        series.append(ChartSeries(
            name=s["name"],
            display_name=s["display_name"],
            data=s[data_key],
        ))

```

```

    ))

    return ChartDataResponse(
        temperatures=result["temperatures"],
        series=series,
    )

@router.get("/jobs/{job_id}/download")
async def download_result(job_id: str):
    """下载原始计算文件 (.equi + .mac + .res 的 zip 包) """
    job_dir = settings.work_root / job_id
    if not job_dir.exists():
        raise HTTPException(status_code=404, detail="任务目录不存在")

    files_to_zip = []
    for subdir in ("input", "out"):
        d = job_dir / subdir
        if d.exists():
            for f in d.iterdir():
                if f.is_file() and f.suffix in (".equi", ".mac", ".res"):
                    files_to_zip.append(f)
    if not files_to_zip:
        raise HTTPException(status_code=404, detail="无可下载的文件")

    buf = io.BytesIO()
    with zipfile.ZipFile(buf, "w", zipfile.ZIP_DEFLATED) as zf:
        for p in files_to_zip:
            arcname = f"{p.parent.name}/{p.name}"
            zf.write(p, arcname)
    buf.seek(0)

    return StreamingResponse(
        buf,
        media_type="application/zip",
        headers={
            "Content-Disposition": f'attachment; filename="{job_id}_result.zip"'
        },
    )

@router.get("/jobs/{job_id}/export-csv")
async def export_csv(
    job_id: str,
    value_type: str = Query("gram", description="数据类型"),
    min_gram: float = Query(
        settings.default_species_min_gram,
        ge=0,
        description="按物种最大质量过滤, 单位 g",
    ),

```

```

):
    """导出选中的数据为 csv"""
    result = job_manager.get_parsed_result(job_id)
    if result is None:
        raise HTTPException(status_code=404, detail="结果不存在")

    type_map = {
        "gram": "grams",
        "mole": "moles",
        "wt_pct": "wt_pcts",
        "activity": "activities",
        "mole_fraction": "mole_fractions",
    }
    data_key = type_map.get(value_type, "grams")

    # 只导出达到当前阈值的物种
    significant = filter_species_by_min_gram(result["species"], min_gram)
    significant.sort(key=lambda x: x["max_gram"], reverse=True)

    buf = io.StringIO()
    writer = csv.writer(buf)

    # 表头
    header = ["Temperature (°C)"] + [s["display_name"] for s in significant]
    writer.writerow(header)

    # 数据行
    for i, t in enumerate(result["temperatures"]):
        row = [t]
        for s in significant:
            vals = s[data_key]
            row.append(vals[i] if i < len(vals) else 0)
        writer.writerow(row)

    buf.seek(0)
    return StreamingResponse(
        iter([buf.getvalue()]),
        media_type="text/csv",
        headers={
            "Content-Disposition": f'attachment; filename="{job_id}_{value_type}.csv"'
        },
    )

@router.get("/jobs")
async def list_jobs() -> List[JobListItem]:
    """列出所有任务"""
    return [
        JobListItem(
            job_id=j["job_id"],

```



```

        status=j["status"],
        template_id=j["template_id"],
        created_at=j["created_at"],
    )
    for j in job_manager.list_all()
]

@router.get("/config/info")
async def config_info():
    """返回当前运行模式信息"""
    return {
        "mock_mode": settings.mock_mode,
        "factsage_dir": str(settings.factsage_dir),
        "templates_dir": str(settings.templates_dir),
        "species_filter": {
            "default_min_gram": settings.default_species_min_gram,
            "options": settings.species_min_gram_options,
        },
    }

# ===== File: backend/app/services/db.py =====
# -*- coding: utf-8 -*-
"""SQLite 持久化层 — 任务元数据存储"""
from __future__ import annotations

import sqlite3
from pathlib import Path
from typing import Dict, List, Optional

_SCHEMA = """
CREATE TABLE IF NOT EXISTS jobs (
    job_id          TEXT PRIMARY KEY,
    status          TEXT NOT NULL DEFAULT 'pending',
    template_id     TEXT NOT NULL,
    request         TEXT NOT NULL,
    created_at      TEXT NOT NULL,
    result_path     TEXT,
    error           TEXT
);
CREATE INDEX IF NOT EXISTS idx_jobs_created_at ON jobs(created_at DESC);
"""

class JobDB:
    """轻量 SQLite 封装，同步操作，单连接"""

    def __init__(self, db_path: Path) -> None:
        self._path = db_path
        db_path.parent.mkdir(parents=True, exist_ok=True)

```

```

        self._conn = sqlite3.connect(str(db_path), check_same_thread=False)
        self._conn.row_factory = sqlite3.Row
        self._conn.executescript(_SCHEMA)

    def close(self) -> None:
        self._conn.close()

    def insert(self, job_id: str, status: str, template_id: str,
               request: str, created_at: str) -> None:
        self._conn.execute(
            "INSERT INTO jobs (job_id, status, template_id, request, created_at) "
            "VALUES (?, ?, ?, ?, ?)",
            (job_id, status, template_id, request, created_at),
        )
        self._conn.commit()

    def update_status(self, job_id: str, status: str) -> None:
        self._conn.execute(
            "UPDATE jobs SET status = ? WHERE job_id = ?", (status, job_id)
        )
        self._conn.commit()

    def update_result_path(self, job_id: str, status: str,
                          result_path: str) -> None:
        self._conn.execute(
            "UPDATE jobs SET status = ?, result_path = ? WHERE job_id = ?",
            (status, result_path, job_id),
        )
        self._conn.commit()

    def update_error(self, job_id: str, status: str, error: str) -> None:
        self._conn.execute(
            "UPDATE jobs SET status = ?, error = ? WHERE job_id = ?",
            (status, error, job_id),
        )
        self._conn.commit()

    def get(self, job_id: str) -> Optional[Dict]:
        cur = self._conn.execute("SELECT * FROM jobs WHERE job_id = ?", (job_id,))
        row = cur.fetchone()
        return dict(row) if row else None

    def list_all(self, limit: int = 100) -> List[Dict]:
        cur = self._conn.execute(
            "SELECT * FROM jobs ORDER BY created_at DESC LIMIT ?", (limit,)
        )
        return [dict(r) for r in cur.fetchall()]

    def cleanup_orphans(self) -> int:
        """将 pending/running 状态的孤儿任务标记为 failed"""

```

```

        cur = self._conn.execute(
            "UPDATE jobs SET status = 'failed', error = '服务重启, 任务中断' "
            "WHERE status IN ('pending', 'running')"
        )
        self._conn.commit()
        return cur.rowcount

# ===== File: backend/app/services/job_manager.py =====
# -*- coding: utf-8 -*-
"""任务管理: SQLite 持久化 + 内存队列 + 后台 worker"""
from __future__ import annotations

import asyncio
import json
import logging
import uuid
from datetime import datetime
from pathlib import Path
from typing import Dict, List, Optional

from ..models import JobRequest, JobStatus
from .db import JobDB
from .factsage_runner import run_calculation
from .res_parser import ResParser, parsed_result_to_dict
from .template_renderer import render_job_files

logger = logging.getLogger(__name__)

class JobManager:
    """单例任务管理器: SQLite 持久化, FIFO 队列, 单 worker"""

    def __init__(self, db_path: Path | None = None) -> None:
        from ..config import settings
        self._settings = settings
        self._db = JobDB(db_path or settings.db_path)
        self._db.cleanup_orphans()
        self._queue: asyncio.Queue[str] = asyncio.Queue()
        self._worker_task: Optional[asyncio.Task] = None
        self._parser = ResParser()

    # — 生命周期 —————

    async def start(self) -> None:
        self._worker_task = asyncio.create_task(self._worker())
        logger.info("JobManager worker 已启动")

    async def stop(self) -> None:
        if self._worker_task:
            self._worker_task.cancel()

```

```

        try:
            await self._worker_task
        except asyncio.CancelledError:
            pass
    self._db.close()
    logger.info("JobManager worker 已停止")

# — 公开接口 —————

async def submit(self, request: JobRequest) -> str:
    """提交任务, 返回 job_id"""
    job_id = uuid.uuid4().hex[:8]
    created_at = datetime.now().isoformat(timespec="microseconds")
    self._db.insert(
        job_id=job_id,
        status=JobStatus.pending.value,
        template_id=request.template_id,
        request=request.model_dump_json(),
        created_at=created_at,
    )
    await self._queue.put(job_id)
    logger.info("任务 %s 已入队 (template=%s)", job_id, request.template_id)
    return job_id

def get(self, job_id: str) -> Optional[Dict]:
    """查询单个任务"""
    row = self._db.get(job_id)
    if not row:
        return None
    return self._hydrate(row)

def list_all(self, limit: int = 100) -> List[Dict]:
    """列出所有任务 (按创建时间倒序)"""
    return [self._hydrate(row) for row in self._db.list_all(limit=limit)]

def get_parsed_result(self, job_id: str) -> Optional[Dict]:
    """获取已解析的计算结果 (从磁盘 JSON 加载)"""
    row = self._db.get(job_id)
    if not row or row.get("status") != JobStatus.completed.value:
        return None
    result_path = row.get("result_path")
    if not result_path:
        return None
    p = Path(result_path)
    if not p.exists():
        return None
    with open(p, "r", encoding="utf-8") as f:
        return json.load(f)

# — 内部方法 —————

```

```

@staticmethod
def _hydrate(row: Dict) -> Dict:
    """将 DB 行转换为业务 dict"""
    request = None
    if row.get("request"):
        try:
            request = JobRequest.model_validate_json(row["request"])
        except Exception as exc:
            logger.warning("任务 %s 的 request 反序列化失败: %s", row.get("job_id"), exc)

    status = JobStatus.failed
    if row.get("status"):
        try:
            status = JobStatus(row["status"])
        except Exception:
            pass

    return {
        "job_id": row["job_id"],
        "status": status,
        "template_id": row.get("template_id", ""),
        "request": request,
        "created_at": row["created_at"],
        "result_path": row.get("result_path"),
        "error": row.get("error"),
    }

# — 后台 worker —————
async def _worker(self) -> None:
    while True:
        job_id = await self._queue.get()
        row = self._db.get(job_id)
        if not row:
            self._queue.task_done()
            continue

        self._db.update_status(job_id, JobStatus.running.value)
        logger.info("任务 %s 开始执行", job_id)

        try:
            request = JobRequest.model_validate_json(row["request"])

            # 1. 渲染模板 -> .equi + .mac
            paths = render_job_files(
                job_id,
                request.template_id,
                [{"symbol": e.symbol, "mass_g": e.mass_g} for e in request.elements],
                request.temp_range.model_dump(),
            )

```

```

    )

    # 2. 执行 FactSage → .res (真实模式) 或直接生成 JSON (mock)
    output_path = await run_calculation(job_id, paths)

    if self._settings.mock_mode:
        # mock 模式: factsage_runner 直接生成 parsed_result.json
        result_path = output_path
    else:
        # 真实模式: 解析 .res → ParsedResult → JSON
        parsed = self._parser.parse(output_path)
        result_json = parsed_result_to_dict(parsed)
        result_path = paths["out_dir"] / "parsed_result.json"
        with open(result_path, "w", encoding="utf-8") as f:
            json.dump(result_json, f, ensure_ascii=False)

    self._db.update_result_path(
        job_id, JobStatus.completed.value, str(result_path)
    )
    logger.info("任务 %s 完成", job_id)
except Exception as exc:
    self._db.update_error(job_id, JobStatus.failed.value, str(exc))
    logger.error("任务 %s 失败: %s", job_id, exc, exc_info=True)
finally:
    self._queue.task_done()

# 全局单例
job_manager = JobManager()

# ===== File: backend/app/services/template_renderer.py =====
# -*- coding: utf-8 -*-
"""模板渲染: 简单 {{KEY}} 占位符替换, 生成 .equi 和 .mac 文件"""
from __future__ import annotations

import json
import re
from pathlib import Path
from typing import Any, Dict, List

from ..config import settings

# — 注册表 / 元数据加载 —————

def load_registry() -> dict:
    """加载 templates/registry.json"""
    registry_path = settings.templates_dir / "registry.json"
    with open(registry_path, "r", encoding="utf-8-sig") as f:
        return json.load(f)

```

```

def load_template_meta(template_id: str) -> dict:
    """加载 templates/{template_id}/meta.json"""
    meta_path = settings.templates_dir / template_id / "meta.json"
    if not meta_path.exists():
        raise FileNotFoundError(f"模板 '{template_id}' 的 meta.json 不存在: {meta_path}")
    with open(meta_path, "r", encoding="utf-8-sig") as f:
        return json.load(f)

# — 文件 I/O —————

def _read_text(path: Path) -> str:
    """读取模板文件（兼容 BOM），统一行尾为 \n"""
    raw = path.read_bytes().decode("utf-8-sig")
    raw = raw.replace("\r\r\n", "\n").replace("\r\n", "\n").replace("\r", "\n")
    return raw

def _write_text(path: Path, text: str) -> None:
    """写出文件，统一使用 Windows CRLF 行尾，UTF-8 无 BOM"""
    path.parent.mkdir(parents=True, exist_ok=True)
    text = text.replace("\r\n", "\n").replace("\r", "\n")
    text = text.replace("\n", "\r\n")
    path.write_text(text, encoding="utf-8", newline="")

def _macro_path(path: Path) -> str:
    """FactSage 宏里使用保守的绝对 Windows 路径。"""
    return str(path.resolve()).replace("/", "\\")

def _render_tpl(template_text: str, variables: Dict[str, str]) -> str:
    """简单 {{KEY}} → value 替换"""
    result = template_text
    for key, value in variables.items():
        result = result.replace(f"{{" + key + "}}", str(value))
    remaining = re.findall(r"\{\{(\w+)\}\}", result)
    if remaining:
        raise ValueError(f"模板中存在未替换的占位符: {remaining}")
    return result

def _mass_placeholders(template_text: str) -> list[str]:
    """提取模板中的 MASS_* 元素占位符。"""
    return re.findall(r"\{\{MASS_([A-Z][a-z]?)\}\}", template_text)

def _validate_template_contract(template_id: str, meta: dict, template_text: str) -> None:

```

```

"""确保 meta 元素全集与 .equi.tpl 占位符一一对应。"""
meta_symbols = [e["symbol"] for e in meta.get("supported_elements", [])]
if len(meta_symbols) != len(set(meta_symbols)):
    duplicated = sorted({s for s in meta_symbols if meta_symbols.count(s) > 1})
    raise ValueError(f"模板 '{template_id}' 的 meta.json 存在重复元素: {duplicated}")

placeholders = _mass_placeholders(template_text)
if len(placeholders) != len(set(placeholders)):
    duplicated = sorted({s for s in placeholders if placeholders.count(s) > 1})
    raise ValueError(f"模板 '{template_id}' 的 case.equi.tpl 存在重复占位符: {duplicated}")

meta_set = set(meta_symbols)
placeholder_set = set(placeholders)
missing = sorted(meta_set - placeholder_set)
extra = sorted(placeholder_set - meta_set)
if missing or extra:
    raise ValueError(
        f"模板 '{template_id}' 的 meta 与占位符不一致: "
        f"meta 未出现在模板={missing}, 模板未登记={extra}"
    )

def _build_mass_variables(
    template_id: str,
    meta: dict,
    elements: List[Dict[str, Any]],
) -> Dict[str, str]:
    """用户提交值覆盖 meta 默认值, 缺失元素由后端兜底补齐。"""
    element_defs = meta.get("supported_elements", [])
    supported = {e["symbol"]: e for e in element_defs}
    variables: Dict[str, str] = {}
    seen: set[str] = set()

    for elem in elements:
        symbol = elem["symbol"]
        if symbol in seen:
            raise ValueError(f"模板 '{template_id}' 收到重复元素: {symbol}")
        if symbol not in supported:
            raise ValueError(f"模板 '{template_id}' 不支持元素: {symbol}")
        seen.add(symbol)
        variables[f"MASS_{symbol}"] = str(elem["mass_g"])

    for symbol, elem_def in supported.items():
        key = f"MASS_{symbol}"
        if key not in variables:
            variables[key] = str(elem_def.get("default_g", 1e-10))

    return variables

```



```
def render_job_files(
    job_id: str,
    template_id: str,
    elements: List[Dict[str, Any]],
    temp_range: Dict[str, Any],
) -> Dict[str, Any]:
    """渲染 .equi 和 .mac 文件。

    Args:
        job_id: 任务唯一标识
        template_id: 模板 ID (如 "high_alloy_incl")
        elements: 元素列表, 每项含 {"symbol": "C", "mass_g": 0.35}
        temp_range: 温度范围, 含 start_c / end_c / step_c / pressure_atm

    Returns:
        dict: job_dir, in_dir, out_dir, equi_path, mac_path
    """
    # 1) 创建工作目录
    job_dir = settings.work_root / job_id
    in_dir = job_dir / "input"
    out_dir = job_dir / "out"
    in_dir.mkdir(parents=True, exist_ok=True)
    out_dir.mkdir(parents=True, exist_ok=True)

    # 2) 读取模板元数据和 .equi.tpl 模板
    meta = load_template_meta(template_id)
    tpl_path = settings.templates_dir / template_id / "case.equi.tpl"
    if not tpl_path.exists():
        raise FileNotFoundError(f"模板文件不存在: {tpl_path}")
    tpl_text = _read_text(tpl_path)
    _validate_template_contract(template_id, meta, tpl_text)

    # 3) 构建替换变量
    variables = _build_mass_variables(template_id, meta, elements)
    variables["T_START"] = str(temp_range.get("start_c", 800))
    variables["T_END"] = str(temp_range.get("end_c", 1600))
    variables["T_STEP"] = str(temp_range.get("step_c", 10))
    variables["P_ATM"] = str(temp_range.get("pressure_atm", 1.0))

    # 4) 渲染 .equi
    equi_text = _render_tpl(tpl_text, variables)
    equi_path = in_dir / "case.equi"
    _write_text(equi_path, equi_text)

    # 5) 生成 .mac 文件 (通用格式, 所有模板共享)
    equi_macro_path = _macro_path(equi_path)
    result_macro_path = _macro_path(out_dir / "result.res")
    mac_text = (
```

```

        "VARIABLE %EquiFile %OutFile\r\n"
        "HIDE\r\n"
        "HIDE MACRO\r\n"
        f"%EquiFile = \"{equi_macro_path}\"\\r\n"
        f"%OutFile = \"{result_macro_path}\"\\r\n"
        "\\r\n"
        "OPEN %EquiFile\r\n"
        "CALC\r\n"
        "SAVE %OutFile\r\n"
        "\\r\n"
        "END\r\n"
    )
    mac_path = in_dir / "case.mac"
    # mac_text 已经包含 CRLF, 直接写出
    mac_path.parent.mkdir(parents=True, exist_ok=True)
    mac_path.write_text(mac_text, encoding="utf-8", newline="")

    return {
        "template_id": template_id,
        "job_dir": job_dir,
        "in_dir": in_dir,
        "out_dir": out_dir,
        "equi_path": equi_path,
        "mac_path": mac_path,
    }

# ===== File: backend/app/services/factsage_runner.py =====
# -*- coding: utf-8 -*-
"""FactSage 执行服务: 调用 EquiSage.exe 或生成 mock 数据"""
from __future__ import annotations

import asyncio
import json
import logging
import subprocess
import sys
import time
from pathlib import Path
from typing import Any, Dict

from ..config import settings
from .res_parser import ResParser, parsed_result_to_dict

logger = logging.getLogger(__name__)

async def run_calculation(
    job_id: str, paths: Dict[str, Any]
) -> Path:
    """执行 FactSage 计算, 返回 .res 文件路径。"""

```

```

Args:
    job_id: 任务 ID
    paths: 由 template_renderer.render_job_files 返回的路径字典

Returns:
    result.res 文件的 Path
"""
if settings.mock_mode:
    return await _mock_calculation(paths)
return await _real_calculation(paths)

async def _real_calculation(paths: Dict[str, Any]) -> Path:
    """调用 EquiSage.exe 执行真实计算"""
    loop = asyncio.get_event_loop()
    rc = await loop.run_in_executor(
        None, _run_factsage_blocking, paths
    )
    if rc != 0:
        raise RuntimeError(f"FactSage 退出码: {rc}")

    res_path: Path = paths["out_dir"] / "result.res"
    if not res_path.exists():
        raise FileNotFoundError(f"FactSage 输出未找到: {res_path}")

    return res_path

def _run_factsage_blocking(paths: Dict[str, Any]) -> int:
    """同步调用 EquiSage.exe (在线程池中执行)"""
    exe = settings.factsage_exe
    if not exe.exists():
        raise FileNotFoundError(f"找不到 EquiSage.exe: {exe}")

    mac_path: Path = paths["mac_path"]
    run_log = Path(paths["out_dir"]) / "factsage_run.log"
    cmd = [str(exe), "/EQUILIB", "/MACRO", str(mac_path)]

    startupinfo = None
    if sys.platform == "win32":
        startupinfo = subprocess.STARTUPINFO()
        startupinfo.dwFlags |= subprocess.STARTF_USESHOWWINDOW
        startupinfo.wShowWindow = 0 # SW_HIDE

    run_log.parent.mkdir(parents=True, exist_ok=True)
    with open(run_log, "w", encoding="utf-8") as f:
        f.write(f"cmd={cmd!r}\n")
        f.write(f"cwd={settings.factsage_dir}\n")
        f.write(f"mac_path={mac_path}\n")

```

```

f.write(f"equi_path={paths.get('equi_path')}\n")
f.write(f"out_dir={paths.get('out_dir')}\n")
f.write(f"started_at={time.strftime('%Y-%m-%d %H:%M:%S')}\n")
f.flush()
p = subprocess.Popen(
    cmd,
    cwd=str(settings.factsage_dir),
    startupinfo=startupinfo,
    stdout=f,
    stderr=subprocess.STDOUT,
)
try:
    rc = p.wait(timeout=settings.factsage_timeout)
except subprocess.TimeoutExpired:
    p.kill()
    f.write(f"timeout_after_seconds={settings.factsage_timeout}\n")
    raise
f.write(f"returncode={rc}\n")
f.write(f"finished_at={time.strftime('%Y-%m-%d %H:%M:%S')}\n")
return rc

async def _mock_calculation(paths: Dict[str, Any]) -> Path:
    """Mock 模式: 优先使用 demo/Equi2.res 的真实解析结果。"""
    await asyncio.sleep(settings.mock_delay)

    result_path = paths["out_dir"] / "parsed_result.json"
    result_path.parent.mkdir(parents=True, exist_ok=True)

    demo_res = _find_demo_res(paths.get("template_id"))
    if not demo_res.exists():
        raise FileNotFoundError(f"Mock 结果文件不存在: {demo_res}")

    parsed = ResParser().parse(demo_res)
    result = parsed_result_to_dict(parsed)
    with open(result_path, "w", encoding="utf-8") as f:
        json.dump(result, f, ensure_ascii=False)
    return result_path

def _find_demo_res(template_id: str | None = None) -> Path:
    """按模板优先选择 demo .res; 找不到时回退到通用 Equi2.res。"""
    demo_dirs = [
        settings.templates_dir.parent.parent / "demo",
        settings.templates_dir.parent / "demo",
    ]
    preferred_names: list[str] = []

    if template_id == "bearing_steel_52100":
        preferred_names.append("Equ2222i.res")

```

```

preferred_names.append("Equi2.res")

for demo_dir in demo_dirs:
    for name in preferred_names:
        path = demo_dir / name
        if path.exists():
            return path

return demo_dirs[0] / preferred_names[0]

# ===== File: backend/app/services/res_parser.py =====
# -*- coding: utf-8 -*-
"""
FactSage .res 文件解析器

解析 EQUILIB 模块输出的 .res 文件，提取物种定义、热力学数据和系统元数据。
.res 文件采用固定宽度格式，每行可达 ~127k 字符，包含数百个物种的数据。
"""
from __future__ import annotations

import re
from dataclasses import dataclass, field
from pathlib import Path

__all__ = [
    "SpeciesDef",
    "SpeciesData",
    "ParsedResult",
    "ResParser",
    "filter_species_by_min_gram",
    "parsed_result_to_dict",
    "parse_res_file",
]

# -----
# 相名称映射：内部缩写 → 可读名称
# -----
PHASE_NICKNAMES: dict[str, str] = {
    "FCC": "γ-FCC",
    "BCC": "α-BCC",
    "HCP": "ε-HCP",
    "LIQU": "Liquid",
    "Liqu": "Liquid",
    "CEME": "Cementite",
    "SIGM": "Sigma",
    "GAS": "Gas",
}

```

```

# =====
# 数据类
# =====

@dataclass
class SpeciesDef:
    """物种定义——从 species identifier line 解析而来。"""
    raw_name: str          # 原始名称, 如 FSstel-Al(Liqu#1)
    display_name: str      # 可读名称, 如 Al in Liquid
    category: str          # solution_component / solution_total / pure / element
    phase: str             # 相标识, 如 FSstel-Liqu#1 / PURE / ELEMENT
    db: str                # 数据库来源: FSstel / FactPS / FToxid / ""
    index: int             # 在物种列表中的位置索引 (0-based)

@dataclass
class SpeciesData:
    """单个物种在所有温度步上的数值数据。"""
    definition: SpeciesDef
    temperatures: list[float] = field(default_factory=list)
    grams: list[float] = field(default_factory=list)
    moles: list[float] = field(default_factory=list)
    activities: list[float] = field(default_factory=list)
    mole_fractions: list[float] = field(default_factory=list)
    wt_pcts: list[float] = field(default_factory=list)
    max_gram: float = 0.0

@dataclass
class ParsedResult:
    """解析完成后的顶层结果。"""
    species: list[SpeciesData]
    temperatures: list[float]
    n_steps: int
    n_solutions: int
    version: str
    reactant_summary: str

# =====
# 解析器
# =====

class ResParser:
    """
    FactSage .res 固定宽度格式解析器。

    文件结构 (由标记行驱动, 不依赖硬编码行号):
    - ``ENDF 0`` 标记 header 结束
    - 之后依次为: 反应物摘要、SOLUTIONS、相头、EQUILIB results、VERSION、
    """

```

```

}

/* ---- Element Card Header ---- */
.element-card-header {
  display: flex;
  justify-content: space-between;
  align-items: flex-start;
  flex-wrap: wrap;
  gap: 8px;
}

.element-card-header .card-title {
  margin-bottom: 0;
  padding-bottom: 0;
  border-bottom: none;
}

.total-mass-indicator {
  font-size: 0.85rem;
  font-weight: 600;
  padding: 3px 10px;
  border-radius: 12px;
  white-space: nowrap;
}

.total-mass-indicator.mass-ok {
  background: #eafaf1;
  color: #27ae60;
}

.total-mass-indicator.mass-warn {
  background: #fef9e7;
  color: #f39c12;
}

.total-mass-indicator.mass-bad {
  background: #fef0f0;
  color: #e74c3c;
}

/* ---- Element Inputs ---- */
.element-inputs {
  display: flex;
  flex-direction: column;
  gap: 12px;
  max-height: 480px;
  overflow-y: auto;
  margin-top: 12px;
}

```

```

.element-group {
  border: 1px solid #ecec4;
  border-radius: 8px;
  padding: 8px;
  background: #fcfcff;
}

.element-group-title {
  font-size: 0.82rem;
  font-weight: 700;
  color: #555b72;
  margin-bottom: 6px;
  padding-bottom: 5px;
  border-bottom: 1px solid #ededf5;
}

.element-row {
  display: flex;
  align-items: center;
  gap: 8px;
  padding: 4px 6px;
  border-radius: 6px;
  transition: background 0.15s, opacity 0.2s;
}

.element-row:hover {
  background: #f8f8fc;
}

.element-row.inactive {
  opacity: 0.5;
  background: #f8f8f8;
}

.element-toggle {
  width: 16px;
  height: 16px;
  cursor: pointer;
  accent-color: #4a6cf7;
  flex-shrink: 0;
}

.element-toggle:disabled {
  cursor: not-allowed;
  opacity: 0.6;
}

.element-label {
  width: 36px;
  font-weight: 600;
}

```



```

    font-size: 0.9rem;
    color: #1a1a2e;
    flex-shrink: 0;
}

.element-input {
    flex: 1;
    padding: 5px 8px;
    border: 1px solid #d0d0d0;
    border-radius: 6px;
    font-size: 0.9rem;
    font-family: inherit;
    background: #fafafa;
    transition: border-color 0.2s, box-shadow 0.2s;
}

.element-input:focus {
    outline: none;
    border-color: #4a6cf7;
    box-shadow: 0 0 0 3px rgba(74, 108, 247, 0.12);
    background: #fff;
}

.element-input:disabled {
    background: #eee;
    color: #aaa;
    cursor: not-allowed;
    border-color: #ddd;
}

.element-unit {
    width: 24px;
    font-size: 0.85rem;
    color: #888;
    flex-shrink: 0;
}

/* ---- Temperature Grid ---- */
.temp-grid {
    display: grid;
    grid-template-columns: 1fr 1fr;
    gap: 12px;
}

/* ---- Buttons ---- */
.btn-primary {
    width: 100%;
    padding: 12px;
    background: #4a6cf7;
    color: #fff;
}

```

```

border: none;
border-radius: 8px;
font-size: 1rem;
font-weight: 600;
font-family: inherit;
cursor: pointer;
transition: background 0.2s, transform 0.1s;
}

.btn-primary:hover:not(:disabled) {
  background: #3a5ce5;
  transform: translateY(-1px);
}

.btn-primary:active:not(:disabled) {
  transform: translateY(0);
}

.btn-primary:disabled {
  background: #b0b0b0;
  cursor: not-allowed;
}

.btn-primary.loading {
  position: relative;
  color: transparent;
}

.btn-primary.loading::after {
  content: "";
  position: absolute;
  width: 20px;
  height: 20px;
  top: 50%;
  left: 50%;
  margin: -10px 0 0 -10px;
  border: 3px solid rgba(255, 255, 255, 0.3);
  border-top-color: #fff;
  border-radius: 50%;
  animation: spin 0.6s linear infinite;
}

@keyframes spin {
  to { transform: rotate(360deg); }
}

.btn-export {
  padding: 8px 16px;
  background: #fff;
  color: #4a6cf7;
}

```

```

border: 1px solid #4a6cf7;
border-radius: 6px;
font-size: 0.85rem;
font-family: inherit;
cursor: pointer;
transition: all 0.2s;
}

.btn-export:hover {
  background: #4a6cf7;
  color: #fff;
}

.export-bar {
  display: flex;
  gap: 12px;
  justify-content: flex-end;
}

/* ---- Status ---- */
.status-indicator {
  margin-top: 12px;
  display: flex;
  align-items: center;
  gap: 8px;
  font-size: 0.9rem;
}

.status-badge {
  display: inline-block;
  width: 10px;
  height: 10px;
  border-radius: 50%;
  flex-shrink: 0;
}

.status-badge.pending {
  background: #f0ad4e;
  animation: pulse-glow 1.5s ease-in-out infinite;
}

.status-badge.running {
  background: #5bc0de;
  animation: pulse-glow 1s ease-in-out infinite;
}

.status-badge.completed {
  background: #5cb85c;
}

```

```

.status-badge.failed {
  background: #d9534f;
}

.error-msg {
  margin-top: 8px;
  padding: 8px 12px;
  background: #fef0f0;
  border: 1px solid #f5c6c6;
  border-radius: 6px;
  color: #c0392b;
  font-size: 0.85rem;
  word-break: break-all;
}

/* ---- Calculation Overlay ---- */
.calculation-overlay {
  position: fixed;
  inset: 0;
  z-index: 10000;
  display: flex;
  align-items: center;
  justify-content: center;
  padding: 24px;
  background: rgba(13, 17, 31, 0.72);
  backdrop-filter: blur(3px);
}

.calculation-overlay[hidden] {
  display: none;
}

body.calculation-overlay-open {
  overflow: hidden;
}

.calculation-overlay-card {
  width: min(420px, 100%);
  padding: 32px 28px;
  border-radius: 12px;
  background: #fff;
  box-shadow: 0 18px 48px rgba(0, 0, 0, 0.28);
  text-align: center;
}

.calculation-spinner {
  width: 52px;
  height: 52px;
  margin: 0 auto 18px;
  border: 5px solid #e2e6ff;
}

```

```

    border-top-color: #4a6cf7;
    border-radius: 50%;
    animation: spin 0.8s linear infinite;
}

.calculation-overlay-title {
    color: #1a1a2e;
    font-size: 1.3rem;
    font-weight: 700;
    line-height: 1.35;
}

.calculation-overlay-status {
    margin-top: 10px;
    color: #4a6cf7;
    font-size: 1rem;
    font-weight: 600;
}

.calculation-overlay-job {
    margin-top: 12px;
    color: #666;
    font-size: 0.85rem;
    word-break: break-all;
}

.calculation-overlay-job span {
    font-family: "Cascadia Code", "Consolas", monospace;
}

.calculation-overlay-hint {
    margin-top: 10px;
    color: #999;
    font-size: 0.82rem;
}

/* ---- History ---- */
.history-list {
    max-height: 200px;
    overflow-y: auto;
    display: flex;
    flex-direction: column;
    gap: 4px;
}

.history-item {
    display: flex;
    justify-content: space-between;
    align-items: center;
    padding: 8px 12px;

```

```

    border-radius: 6px;
    cursor: pointer;
    transition: background 0.15s;
    font-size: 0.85rem;
    border: 1px solid transparent;
}

.history-item:hover {
    background: #f0f2ff;
    border-color: #d0d8ff;
}

.history-item.active {
    background: #e8ecff;
    border-color: #4a6cf7;
}

.history-item .job-id {
    font-family: "Cascadia Code", "Consolas", monospace;
    color: #555;
    font-size: 0.8rem;
}

.history-item .job-time {
    color: #999;
    font-size: 0.75rem;
}

.history-item .job-status {
    font-size: 0.75rem;
    padding: 2px 8px;
    border-radius: 10px;
    font-weight: 500;
}

.history-item .job-status.completed {
    background: #eafaf1;
    color: #27ae60;
}

.history-item .job-status.failed {
    background: #fef0f0;
    color: #e74c3c;
}

.history-item .job-status.running {
    background: #ebf5fb;
    color: #2e86c1;
}

```

```

.history-item .job-status.pending {
  background: #fef9e7;
  color: #f39c12;
}

/* ---- Chart ---- */
.chart-container {
  width: 100%;
  height: 500px;
  background: #fafafa;
  border-radius: 6px;
}

/* ---- Species Panel ---- */
.species-header {
  display: flex;
  justify-content: space-between;
  align-items: flex-start;
  flex-wrap: wrap;
  gap: 8px;
}

.species-header .card-title {
  margin-bottom: 0;
  padding-bottom: 0;
  border-bottom: none;
}

.species-controls {
  display: flex;
  align-items: center;
  gap: 12px;
  flex-wrap: wrap;
  justify-content: flex-end;
}

.value-type-selector {
  display: flex;
  align-items: center;
  gap: 6px;
  font-size: 0.85rem;
}

.value-type-selector select {
  padding: 4px 8px;
  border: 1px solid #d0d0d0;
  border-radius: 4px;
  font-family: inherit;
  font-size: 0.85rem;
  cursor: pointer;
}

```

```

}

.species-panel {
  max-height: 360px;
  overflow-y: auto;
  margin-top: 12px;
}

.species-group {
  margin-bottom: 16px;
}

.species-group-header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 6px 0;
  border-bottom: 1px solid #eee;
  margin-bottom: 6px;
}

.species-group-title {
  font-size: 0.9rem;
  font-weight: 600;
  color: #1a1a2e;
}

.toggle-all-btn {
  font-size: 0.75rem;
  color: #4a6cf7;
  background: none;
  border: none;
  cursor: pointer;
  font-family: inherit;
  padding: 2px 6px;
  border-radius: 4px;
  transition: background 0.15s;
}

.toggle-all-btn:hover {
  background: #f0f2ff;
}

.species-item {
  display: flex;
  align-items: center;
  gap: 8px;
  padding: 4px 8px;
  border-radius: 4px;
  transition: background 0.1s;
}

```



```

}

.species-item:hover {
  background: #f8f8fc;
}

.species-item input[type="checkbox"] {
  width: 16px;
  height: 16px;
  cursor: pointer;
  accent-color: #4a6cf7;
}

.species-item .species-name {
  flex: 1;
  font-size: 0.85rem;
}

.species-item .species-max {
  font-size: 0.75rem;
  color: #999;
  font-family: "Cascadia Code", "Consolas", monospace;
}

/* ---- Scrollbar ---- */
.element-inputs::-webkit-scrollbar,
.species-panel::-webkit-scrollbar,
.history-list::-webkit-scrollbar {
  width: 6px;
}

.element-inputs::-webkit-scrollbar-track,
.species-panel::-webkit-scrollbar-track,
.history-list::-webkit-scrollbar-track {
  background: transparent;
}

.element-inputs::-webkit-scrollbar-thumb,
.species-panel::-webkit-scrollbar-thumb,
.history-list::-webkit-scrollbar-thumb {
  background: #ccc;
  border-radius: 3px;
}

.element-inputs::-webkit-scrollbar-thumb:hover,
.species-panel::-webkit-scrollbar-thumb:hover,
.history-list::-webkit-scrollbar-thumb:hover {
  background: #aaa;
}

```

```

/* ---- Responsive ---- */
@media (max-width: 1024px) {
  .main-container {
    flex-direction: column;
  }

  .panel-left,
  .panel-right {
    width: 100%;
    min-width: unset;
  }

  .chart-container {
    height: 400px;
  }
}

@media (max-width: 600px) {
  .header {
    padding: 0 12px;
    height: 48px;
  }

  .header-title {
    font-size: 1rem;
  }

  .temp-grid {
    grid-template-columns: 1fr;
  }

  .chart-container {
    height: 300px;
  }

  .export-bar {
    flex-wrap: wrap;
  }

  .species-header {
    flex-direction: column;
  }

  .species-controls {
    justify-content: flex-start;
  }

  .calculation-overlay {
    padding: 16px;
  }
}

```

```

    .calculation-overlay-card {
      padding: 28px 20px;
    }

    .calculation-overlay-title {
      font-size: 1.15rem;
    }
  }

/* ===== File: frontend/js/app.js ===== */
/* =====
FactSage 夹杂物/析出相计算系统 - 前端逻辑
===== */

// ---- 状态 ----
let currentJobId = null;
let speciesList = [];
let chartInstance = null;
let pollingTimer = null;
let currentTemplateId = 'high_alloy_incl';

const DEFAULT_TEMPLATE_ID = 'high_alloy_incl';
const DEFAULT_SPECIES_MIN_GRAM = 1e-8;

const STATUS_LABELS = {
  pending: '等待中',
  running: '计算中',
  completed: '已完成',
  failed: '失败',
};

const OVERLAY_STATUS_TEXT = {
  submitting: '提交任务中',
  pending: '等待计算队列',
  running: 'FactSage 计算中',
  loading: '正在加载结果',
};

// ---- 初始化 ----
document.addEventListener('DOMContentLoaded', () => {
  chartInstance = echarts.init(document.getElementById('chart'));
  normalizeTemplateCard();
  loadTemplates();
  loadHistory();
  checkMockMode();

  window.addEventListener('resize', () => {
    if (chartInstance) chartInstance.resize();
  });
});

```

```

});

function normalizeTemplateCard() {
  if (document.getElementById('templateName') && document.getElementById('templateSelect')) return;

  const legacySelect = document.getElementById('templateSelect');
  const card = legacySelect ? legacySelect.closest('.card') : null;
  if (!card) return;

  card.innerHTML = `
    <h2 class="card-title">计算体系</h2>
    <select id="templateSelect" class="input-full template-select" autocomplete="off"
      onchange="onTemplateChange(this.value)" disabled>
      <option value="">正在加载计算体系...</option>
    </select>
    <div id="templateName" class="template-name">探索模式</div>
    <p id="templateDesc" class="template-desc"></p>
  `;
}

// =====
// API 调用
// =====

async function apiFetch(url, options) {
  try {
    const resp = await fetch(url, options);
    if (!resp.ok) {
      const text = await resp.text();
      let msg;
      try {
        const json = JSON.parse(text);
        msg = json.detail || json.message || text;
      } catch {
        msg = text;
      }
      throw new Error(`请求失败 (${resp.status}): ${msg}`);
    }
    return resp;
  } catch (err) {
    if (err.name === 'TypeError' && err.message.includes('fetch')) {
      throw new Error('无法连接到服务器，请确认后端服务是否已启动。');
    }
    throw err;
  }
}

// 检查是否为模拟模式
async function checkMockMode() {

```

```

    try {
      const resp = await apiFetch('/api/config/info');
      const data = await resp.json();
      if (data.mock_mode) {
        document.getElementById('mockBadge').style.display = '';
      }
      renderSpeciesThresholdOptions(data.species_filter);
    } catch {
      // 静默失败，不阻塞主流程
    }
  }
}

function renderSpeciesThresholdOptions(speciesFilter) {
  const select = document.getElementById('speciesMinGram');
  if (!select || !speciesFilter || !Array.isArray(speciesFilter.options)) return;

  const defaultValue = String(speciesFilter.default_min_gram ?? DEFAULT_SPECIES_MIN_GRAM);
  select.innerHTML = speciesFilter.options
    .map(value => {
      const text = `${formatNumber(value)} g`;
      const selected = String(value) === defaultValue ? 'selected' : '';
      return `<option value="${value}" ${selected}>${text}</option>`;
    })
    .join('');
}

// 加载模板列表
async function loadTemplates() {
  try {
    const resp = await apiFetch('/api/templates');
    const templates = await resp.json();
    const select = document.getElementById('templateSelect');
    if (select) {
      select.innerHTML = templates
        .map(t => `<option value="${t.id}">${t.name}</option>`)
        .join('');
      const hasDefault = templates.some(t => t.id === currentTemplateId);
      currentTemplateId = hasDefault ? currentTemplateId : (templates[0]?.id || DEFAULT_TEMPLATE_ID);
      select.value = currentTemplateId;
      select.disabled = false;
    }
    await loadTemplateMeta(currentTemplateId);
  } catch (err) {
    showError('加载模板列表失败: ' + err.message);
    await loadTemplateMeta(currentTemplateId);
  }
}

// 加载模板元信息，填充元素输入
async function loadTemplateMeta(templateId) {

```

```

try {
  const resp = await apiFetch(`/api/templates/${templateId}`);
  const meta = await resp.json();
  currentTemplateId = templateId;
  const select = document.getElementById('templateSelect');
  if (select) select.value = templateId;

  // 填充元素输入
  renderElementInputs(meta.supported_elements);

  // 填充默认温度/压力
  if (meta.default_temp) {
    document.getElementById('tempStart').value = meta.default_temp.start_c ?? '';
    document.getElementById('tempEnd').value = meta.default_temp.end_c ?? '';
    document.getElementById('tempStep').value = meta.default_temp.step_c ?? '';
  }
  if (meta.default_pressure_atm != null) {
    document.getElementById('pressure').value = meta.default_pressure_atm;
  }

  // 体系说明
  const nameEl = document.getElementById('templateName');
  const descEl = document.getElementById('templateDesc');
  if (nameEl) nameEl.textContent = meta.name || '探索模式';
  if (descEl) descEl.textContent = meta.description || '';

  // 启用提交按钮
  document.getElementById('submitBtn').disabled = false;
} catch (err) {
  showError('加载模板详情失败: ' + err.message);
}
}

async function onTemplateChange(templateId) {
  if (!templateId || templateId === currentTemplateId) return;

  currentJobId = null;
  speciesList = [];
  hideError();
  clearStatus();
  renderSpeciesList([]);
  if (chartInstance) chartInstance.clear();
  document.getElementById('submitBtn').disabled = true;
  await loadTemplateMeta(templateId);
}

// 提交计算
async function submitCalculation() {
  // 收集全部元素数据 (active 发用户值, inactive 发 1E-10)
  const elementRows = document.querySelectorAll('.element-row');

```

```

const elements = [];
for (const row of elementRows) {
  const toggle = row.querySelector('.element-toggle');
  const input = row.querySelector('.element-input');
  const symbol = input.dataset.symbol;

  if (toggle.checked) {
    const val = parseFloat(input.value);
    if (isNaN(val) || val <= 0) {
      showError(`元素 ${symbol} 已启用但未填写有效质量。`);
      return;
    }
    elements.push({ symbol, mass_g: val });
  } else {
    elements.push({ symbol, mass_g: 1e-10 });
  }
}

if (elements.length === 0) {
  showError('请至少输入一个元素的质量。');
  return;
}

// 收集温度/压力
const startC = parseFloat(document.getElementById('tempStart').value);
const endC = parseFloat(document.getElementById('tempEnd').value);
const stepC = parseFloat(document.getElementById('tempStep').value);
const pressure = parseFloat(document.getElementById('pressure').value);

if (isNaN(startC) || isNaN(endC) || isNaN(stepC)) {
  showError('请完整填写温度范围（起始、终止、步长）。');
  return;
}

if (startC >= endC) {
  showError('起始温度必须小于终止温度。');
  return;
}

if (stepC <= 0) {
  showError('温度步长必须大于 0。');
  return;
}

const body = {
  template_id: currentTemplateId,
  elements: elements,
  temp_range: {
    start_c: startC,
    end_c: endC,
  },
};

```

```

        step_c: stepC,
        pressure_atm: isNaN(pressure) ? 1 : pressure,
    },
    );
};

// 禁用按钮，显示加载
const btn = document.getElementById('submitBtn');
btn.disabled = true;
btn.classList.add('loading');
btn.textContent = '计算中...';
hideError();
showCalculationOverlay(null, 'submitting');

try {
    const resp = await apiFetch('/api/calculate', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(body),
    });
    const result = await resp.json();
    currentJobId = result.job_id;
    showCalculationOverlay(result.job_id, result.status);
    updateStatus(result.status);
    pollJobStatus(result.job_id);
} catch (err) {
    showError('提交计算失败: ' + err.message);
    resetSubmitBtn();
    hideCalculationOverlay();
}

}

// 轮询任务状态
function pollJobStatus(jobId) {
    if (pollingTimer) clearInterval(pollingTimer);

    pollingTimer = setInterval(async () => {
        try {
            const resp = await apiFetch(`/api/jobs/${jobId}`);
            const job = await resp.json();
            updateStatus(job.status, job.error);
            setCalculationOverlayStatus(job.status);

            if (job.status === 'completed') {
                clearInterval(pollingTimer);
                pollingTimer = null;
                resetSubmitBtn();
                setCalculationOverlayStatus('loading');
            }
            try {
                await loadSpeciesList(jobId);
                loadHistory();
            }
        }
    }, 1000);
}

```



```

        } finally {
            hideCalculationOverlay();
        }
    } else if (job.status === 'failed') {
        clearInterval(pollingTimer);
        pollingTimer = null;
        resetSubmitBtn();
        hideCalculationOverlay();
        showError(job.error || '计算失败, 未知错误。');
        loadHistory();
    }
} catch (err) {
    clearInterval(pollingTimer);
    pollingTimer = null;
    resetSubmitBtn();
    hideCalculationOverlay();
    showError('查询任务状态失败: ' + err.message);
}
}, 2000);
}

// 加载物种列表
async function loadSpeciesList(jobId) {
    try {
        const minGram = getSpeciesMinGram();
        const params = new URLSearchParams();
        params.set('min_gram', String(minGram));

        const resp = await apiFetch(`/api/jobs/${jobId}/species?${params.toString()}`);
        speciesList = await resp.json();
        renderSpeciesList(speciesList);

        // 自动选中 max_gram 最大的前 5 个
        const sorted = [...speciesList].sort((a, b) => b.max_gram - a.max_gram);
        const topNames = sorted.slice(0, 5).map(s => s.name);
        topNames.forEach(name => {
            const cb = document.querySelector(`input[data-species="${name}"]`);
            if (cb) cb.checked = true;
        });

        await reloadChart();
    } catch (err) {
        showError('加载物种列表失败: ' + err.message);
    }
}

// 加载图表数据
async function loadChartData(jobId, speciesNames, valueType) {
    if (!speciesNames.length) {
        chartInstance.clear();
    }
}

```

```

        chartInstance.setOption({
            title: {
                text: '请选择至少一个物种',
                left: 'center',
                top: 'center',
                textStyle: { color: '#aaa', fontSize: 16 },
            },
        });
    };
    return;
}

try {
    const params = new URLSearchParams();
    params.set('species_names', speciesNames.join(','));
    params.set('value_type', valueType);

    const resp = await apiFetch(`/api/jobs/${jobId}/chart-data?${params.toString()}`);
    const data = await resp.json();
    updateChart(data, valueType);
} catch (err) {
    showError('加载图表数据失败: ' + err.message);
}
}

// 加载历史记录
async function loadHistory() {
    try {
        const resp = await apiFetch('/api/jobs');
        const jobs = await resp.json();
        const container = document.getElementById('historyList');

        if (!jobs.length) {
            container.innerHTML = '<p class="hint-text">暂无历史记录</p>';
            return;
        }

        // 按创建时间倒序
        jobs.sort((a, b) => {
            if (!a.created_at || !b.created_at) return 0;
            return new Date(b.created_at) - new Date(a.created_at);
        });

        container.innerHTML = jobs.map(job => {
            const statusLabel = STATUS_LABELS[job.status] || job.status;
            const time = job.created_at ? formatTime(job.created_at) : '';
            const isActive = job.job_id === currentJobId ? ' active' : '';
            return `
                <div class="history-item${isActive}" onclick="loadJob('${job.job_id}')" title="${job.job_id}">
                    <div>
                        <span class="job-id">${shortenId(job.job_id)}</span>
            `;
        });
    }
}

```

```

        <span class="job-time">${time}</span>
      </div>
      <span class="job-status ${job.status}">${statusLabel}</span>
    </div>
  `;
  }).join('');
} catch {
  // 静默失败
}
}

// 点击历史条目加载任务
async function loadJob(jobId) {
  currentJobId = jobId;
  hideError();

  try {
    const resp = await apiFetch(`/api/jobs/${jobId}`);
    const job = await resp.json();
    updateStatus(job.status, job.error);

    if (job.status === 'completed') {
      await loadSpeciesList(jobId);
    } else if (job.status === 'running' || job.status === 'pending') {
      pollJobStatus(jobId);
    }

    loadHistory(); // 刷新高亮
  } catch (err) {
    showError('加载任务失败: ' + err.message);
  }
}

// =====
// UI 渲染
// =====

const ELEMENT_GROUP_LABELS = {
  base: '基体 / 间隙元素',
  major: '主合金元素',
  micro: '微合金元素',
  trace: '痕量 / 杂质元素',
};

const ELEMENT_GROUP_ORDER = ['base', 'major', 'micro', 'trace'];

// 渲染元素输入 (带 checkbox 开关)
function renderElementInputs(elements) {
  const container = document.getElementById('elementInputs');

```

```

if (!elements || !elements.length) {
  container.innerHTML = '<p class="hint-text">该模板无需配置元素</p>';
  return;
}

const groups = {};
elements.forEach(el => {
  const group = el.group || 'trace';
  if (!groups[group]) groups[group] = [];
  groups[group].push(el);
});

const orderedGroups = [
  ...ELEMENT_GROUP_ORDER,
  ...Object.keys(groups).filter(g => !ELEMENT_GROUP_ORDER.includes(g)),
];

container.innerHTML = orderedGroups
  .filter(group => groups[group] && groups[group].length)
  .map(group => `
    <div class="element-group" data-group="${group}">
      <div class="element-group-title">${ELEMENT_GROUP_LABELS[group] || group}</div>
      <div>${groups[group].map(renderElementRow).join('')}</div>
    </div>
  `).join('');

updateTotalMass();
}

function renderElementRow(el) {
  const isActive = el.active !== false;
  const isRequired = !!el.required;
  const checkedAttr = isActive ? 'checked' : '';
  const disabledCb = isRequired ? 'disabled' : '';
  const disabledInput = isActive ? '' : 'disabled';
  const rowClass = isActive ? 'element-row' : 'element-row inactive';
  const displayVal = el.default_g ?? '';

  return `
    <div class="${rowClass}" data-symbol="${el.symbol}">
      <input type="checkbox"
        class="element-toggle"
        data-symbol="${el.symbol}"
        data-required="${isRequired}"
        data-default-g="${el.default_g ?? 1e-10}"
        ${checkedAttr} ${disabledCb}
        onchange="onElementToggle(this)"
        title="${isRequired ? '必选元素, 不可关闭' : '启用/禁用此元素'}"
      >
      <span class="element-label">${el.symbol}</span>
    </div>
  `;
}

```

```

        <input type="number"
            class="element-input"
            data-symbol="${el.symbol}"
            data-required="${isRequired}"
            value="${displayVal}"
            step="any"
            placeholder="质量"
            ${disabledInput}
            oninput="updateTotalMass()"
        >
    <span class="element-unit">g</span>
</div>
`;
}

// 元素开关切换
function onElementToggle(checkbox) {
    const symbol = checkbox.dataset.symbol;
    const row = checkbox.closest('.element-row');
    const input = row.querySelector('.element-input');

    if (checkbox.checked) {
        row.classList.remove('inactive');
        input.disabled = false;
        // 恢复到默认值（如果当前是 1e-10）
        const val = parseFloat(input.value);
        if (val <= 1e-9) {
            input.value = checkbox.dataset.defaultG;
        }
    } else {
        row.classList.add('inactive');
        input.disabled = true;
        input.value = 1e-10;
    }
    updateTotalMass();
}

// 更新总质量显示
function updateTotalMass() {
    const indicator = document.getElementById('totalMassIndicator');
    const valueSpan = document.getElementById('totalMassValue');
    if (!indicator || !valueSpan) return;

    const rows = document.querySelectorAll('.element-row');
    if (!rows.length) {
        indicator.style.display = 'none';
        return;
    }

    let total = 0;

```

```

rows.forEach(row => {
  const toggle = row.querySelector('.element-toggle');
  if (toggle && toggle.checked) {
    const input = row.querySelector('.element-input');
    const val = parseFloat(input.value);
    if (!isNaN(val) && val > 0.001) total += val;
  }
});

valueSpan.textContent = formatNumber(total);
indicator.style.display = '';

// 颜色指示: 接近 100g 为绿色, 偏差大为红色
const deviation = Math.abs(total - 100);
if (deviation <= 1) {
  indicator.className = 'total-mass-indicator mass-ok';
} else if (deviation <= 5) {
  indicator.className = 'total-mass-indicator mass-warn';
} else {
  indicator.className = 'total-mass-indicator mass-bad';
}
}

// 渲染物种列表 (按 category 分组)
function renderSpeciesList(species) {
  const panel = document.getElementById('speciesPanel');

  if (!species || !species.length) {
    panel.innerHTML = '<p class="hint-text">无物种数据</p>';
    return;
  }

  const categoryLabels = {
    solution_total: '溶液相 (Solution Phases)',
    pure: '析出相/纯物质 (Precipitates)',
    solution_component: '溶液组分 (Solution Components)',
    element: '元素 (Elements)',
  };

  // 按 category 分组
  const groups = {};
  species.forEach(s => {
    const cat = s.category || 'other';
    if (!groups[cat]) groups[cat] = [];
    groups[cat].push(s);
  });

  // 每组按 max_gram 降序排列
  Object.values(groups).forEach(arr =>
    arr.sort((a, b) => b.max_gram - a.max_gram)
  );
}

```

```

);

// 渲染顺序: 溶液相总量 → 析出相 → 溶液组分 → 元素
const order = ['solution_total', 'pure', 'solution_component', 'element'];
const allCats = [...order, ...Object.keys(groups).filter(c => !order.includes(c))];

panel.innerHTML = allCats
  .filter(cat => groups[cat] && groups[cat].length)
  .map(cat => {
    const label = categoryLabels[cat] || cat;
    const items = groups[cat];
    return `
      <div class="species-group" data-category="${cat}">
        <div class="species-group-header">
          <span class="species-group-title">${label}</span>
          <button class="toggle-all-btn" onclick="toggleGroup('${cat}')">全选/全不选</button>
        </div>
        ${items.map(s => `
          <label class="species-item">
            <input type="checkbox"
              data-species="${s.name}"
              onchange="onSpeciesChange()"
            >
            <span class="species-name">${s.display_name || s.name}</span>
            <span class="species-max">max: ${formatNumber(s.max_gram)} g</span>
          </label>
        `).join('')}
      </div>
    `;
  })
  .join('');
}

// 更新图表
function updateChart(chartData, valueType) {
  if (!chartData || !chartData.series || !chartData.series.length) {
    chartInstance.clear();
    chartInstance.setOption({
      title: {
        text: '无数据',
        left: 'center',
        top: 'center',
        textStyle: { color: '#aaa', fontSize: 16 },
      },
    });
    return;
  }

  const option = buildChartOption(chartData, valueType);
  chartInstance.clear();
  chartInstance.setOption(option);
}

```

```

}

// 构建 ECharts 配置
function buildChartOption(data, valueType) {
  const yAxisNames = {
    gram: '质量 (g)',
    mole: '摩尔 (mol)',
    'wt%': '质量分数 (wt%)',
    activity: '活度',
    mole_fraction: '摩尔分数',
  };

  return {
    title: { text: '析出相/夹杂物 vs 温度', left: 'center', textStyle: { fontSize: 15 } },
    tooltip: {
      trigger: 'axis',
      confine: true,
      formatter: function (params) {
        if (!params.length) return '';
        const xValue = Array.isArray(params[0].value) ? params[0].value[0] : params[0].axisValue;
        let html = `<strong>${formatNumber(xValue)} °C</strong><br>`;
        params.forEach(p => {
          const yValue = Array.isArray(p.value) ? p.value[1] : p.value;
          html += `${p.marker} ${p.seriesName}: <strong>${formatNumber(yValue)}</strong><br>`;
        });
        return html;
      },
    },
    legend: { type: 'scroll', bottom: 0 },
    grid: { left: 80, right: 30, top: 60, bottom: 80 },
    xAxis: {
      type: 'value',
      name: '温度 (°C)',
      nameLocation: 'middle',
      nameGap: 30,
    },
    yAxis: {
      type: 'value',
      name: yAxisNames[valueType] || valueType,
      nameLocation: 'middle',
      nameGap: 55,
    },
    dataZoom: [
      { type: 'inside' },
      { type: 'slider', bottom: 30 },
    ],
    toolbox: {
      feature: {
        saveAsImage: { title: '保存图片' },
        dataView: { title: '数据视图', readOnly: true },
      },
    },
  };
}

```



```

        restore: { title: '还原' },
    },
    },
    series: data.series.map(s => ({
        name: s.display_name || s.name,
        type: 'line',
        data: data.temperatures.map((temp, idx) => [temp, s.data[idx] ?? 0]),
        smooth: false,
        showSymbol: false,
        emphasis: { focus: 'series' },
    })),
    })),
    });
}

// 更新状态指示器
function updateStatus(status, error) {
    const container = document.getElementById('statusIndicator');
    const badge = document.getElementById('statusBadge');
    const text = document.getElementById('statusText');

    container.style.display = 'flex';
    badge.className = 'status-badge ' + status;
    text.textContent = STATUS_LABELS[status] || status;

    if (status === 'failed' && error) {
        showError(error);
    }
}

function clearStatus() {
    const container = document.getElementById('statusIndicator');
    const badge = document.getElementById('statusBadge');
    const text = document.getElementById('statusText');
    if (!container || !badge || !text) return;

    container.style.display = 'none';
    badge.className = 'status-badge';
    text.textContent = '';
}

// =====
// 事件处理
// =====

// 物种勾选变化 → 重载图表
function onSpeciesChange() {
    reloadChart();
}

// 数值类型变化 → 重载图表

```

```

function onValueTypeChange() {
    reloadChart();
}

// 物种最小质量阈值变化 → 重新加载物种列表
async function onSpeciesThresholdChange() {
    if (!currentJobId) return;
    await loadSpeciesList(currentJobId);
}

// 重载图表（收集选中物种 + 数值类型）
async function reloadChart() {
    if (!currentJobId) return;

    const checked = document.querySelectorAll('input[data-species]:checked');
    const names = Array.from(checked).map(cb => cb.dataset.species);
    const valueType = document.getElementById('valueType').value;

    await loadChartData(currentJobId, names, valueType);
}

// 全选/全不选某分组
function toggleGroup(category) {
    const group = document.querySelector(`.species-group[data-category="${category}"]`);
    if (!group) return;

    const checkboxes = group.querySelectorAll('input[type="checkbox"]');
    const allChecked = Array.from(checkboxes).every(cb => cb.checked);

    checkboxes.forEach(cb => { cb.checked = !allChecked; });
    reloadChart();
}

// =====
// 导出
// =====

function exportCSV() {
    if (!currentJobId) {
        showError('请先完成一次计算。');
        return;
    }
    const params = new URLSearchParams();
    params.set('min_gram', String(getSpeciesMinGram()));
    params.set('value_type', document.getElementById('valueType').value);
    window.open(`/api/jobs/${currentJobId}/export-csv?${params.toString()}`);
}

function downloadFiles() {
    if (!currentJobId) {

```

```

        showError('请先完成一次计算。');
        return;
    }
    window.open(`/api/jobs/${currentJobId}/download`);
}

function exportPNG() {
    if (!chartInstance) {
        showError('图表尚未初始化。');
        return;
    }
    const url = chartInstance.getDataURL({
        type: 'png',
        pixelRatio: 2,
        backgroundColor: '#fff',
    });
    const a = document.createElement('a');
    a.href = url;
    a.download = `factsage_chart_${currentJobId || 'unknown'}.png`;
    a.click();
}

// =====
// 工具函数
// =====

function resetSubmitBtn() {
    const btn = document.getElementById('submitBtn');
    btn.disabled = false;
    btn.classList.remove('loading');
    btn.textContent = '开始计算';
}

function showCalculationOverlay(jobId, status) {
    const overlay = document.getElementById('calculationOverlay');
    const jobEl = document.getElementById('calculationOverlayJobId');
    if (!overlay || !jobEl) return;

    jobEl.textContent = jobId || '等待生成';
    setCalculationOverlayStatus(status || 'submitting');
    document.body.classList.add('calculation-overlay-open');
    overlay.hidden = false;
    overlay.setAttribute('aria-hidden', 'false');
    overlay.focus();
}

function hideCalculationOverlay() {
    const overlay = document.getElementById('calculationOverlay');
    if (!overlay) return;

```

```

    overlay.hidden = true;
    overlay.setAttribute('aria-hidden', 'true');
    document.body.classList.remove('calculation-overlay-open');
}

function setCalculationOverlayStatus(status) {
    const statusEl = document.getElementById('calculationOverlayStatus');
    if (!statusEl) return;

    statusEl.textContent = OVERLAY_STATUS_TEXT[status] || STATUS_LABELS[status] || '处理中';
}

function showError(msg) {
    const el = document.getElementById('errorMsg');
    el.textContent = msg;
    el.style.display = '';
}

function hideError() {
    document.getElementById('errorMsg').style.display = 'none';
}

function shortenId(id) {
    if (!id) return '';
    return id.length > 12 ? id.slice(0, 8) + '...' : id;
}

function formatTime(isoStr) {
    try {
        const d = new Date(isoStr);
        const pad = n => String(n).padStart(2, '0');
        return `${pad(d.getMonth() + 1)}-${pad(d.getDate())} ${pad(d.getHours())}:${pad(d.getMinutes())}`;
    } catch {
        return '';
    }
}

function formatNumber(val) {
    if (val == null || isNaN(val)) return '-';
    if (Math.abs(val) < 0.001 && val !== 0) return val.toExponential(3);
    if (Math.abs(val) >= 10000) return val.toExponential(3);
    return parseFloat(val.toFixed(4)).toString();
}

function getSpeciesMinGram() {
    const el = document.getElementById('speciesMinGram');
    const val = el ? parseFloat(el.value) : DEFAULT_SPECIES_MIN_GRAM;
    return Number.isFinite(val) && val >= 0 ? val : DEFAULT_SPECIES_MIN_GRAM;
}

```